jackjackbits and **claude** Close the three holes the #1486 review found (#1488)

132120a · 3 months ago

94 lines (60 loc) · 6.22 KB

Preview Code Blame Raw

Verifying bitchat

This document is about a specific risk: getting a copy of bitchat that someone has modified.

It matters because the repository has been the target of takedown demands. When a repository or a releases page becomes unavailable, mirrors appear, and people who need the app during a shutdown install whatever they can find. That is exactly the moment a trojaned build reaches the people with the most to lose. A modified bitchat can log plaintext, ship keys off the device, or weaken the mesh, and it will look and behave normally while doing it.

The honest summary is short. **Source can be verified. Compiled apps cannot, unless they come from the App Store.** Everything below elaborates on that.

Getting the app

In order of how much verification is possible:

1. **The App Store.** Apple verifies the developer signature, and the binary cannot be altered without breaking it. This is the only channel where a compiled build is verifiable end to end, and it is the right recommendation for almost everyone.
2. **Build it yourself from verified source.** See below. Requires a Mac and Xcode, and gives you the strongest guarantee if you can do it.

3. **A compiled build from anywhere else.** Not verifiable. See "Builds from other sources".

Verifying source

Every tagged release has a `SOURCE-MANIFEST.txt` produced by `.github/workflows/source-manifest.yml`. It records the tag, the commit, the git tree hash, and a SHA-256 for every tracked file.

Keep the downloaded manifest *outside* the source tree (say, `/tmp`) — a stray copy inside the checkout would itself trip the completeness checks below. Then check a copy of the source against it:

```
# From the root of the source you obtained, with the manifest at /tmp
grep -v '^#' /tmp/SOURCE-MANIFEST.txt > /tmp/files.sha256
shasum -a 256 -c /tmp/files.sha256
```

Any `FAILED` line means that file differs from the released source. Investigate before building.

That check alone is not enough. `shasum -c` verifies the files the manifest lists and says nothing about files it does not list — and the Xcode project compiles every source file present in the tree automatically, so a hostile mirror can pass the hash check by leaving every listed file intact and *adding* one. Confirm nothing extra is present:

```
# The manifest's path list must match the tree exactly — no missing files, r
grep -v '^#' /tmp/SOURCE-MANIFEST.txt | sed 's/^[0-9a-f]*  //' | LC_ALL=C sc
find . -type f ! -path './.git/*' | sed 's|^\./||' | LC_ALL=C sort > /tmp/ac
diff /tmp/manifest-paths /tmp/actual-paths   # must print nothing
```

In a git checkout the same assurance is one command — it also catches extra files, because they show as untracked. `--ignored` matters: `.gitignore` covers paths like `build/`, plain `git status` would not report a planted file there, and Xcode compiles it all the same:

```
git status --porcelain --ignored      # must print nothing before you build
```

The single value that covers the whole tree is the git tree hash in the manifest header:

```
git rev-parse HEAD^{tree}   # must equal the "tree:" line in the manifest
```

Note the tree hash covers tracked content only; it does not see untracked files sitting in the working directory, which is why the emptiness checks above come first.

The manifest itself carries a provenance attestation tying it to the workflow run that produced it, so a manifest handed to you along with a mirror is checkable too:

```
gh attestation verify /tmp/SOURCE-MANIFEST.txt --repo permissionlesstech/bit
```

That last step is what makes this resistant to a hostile mirror. Without it, whoever gives you the source can give you a matching manifest.

If the manifest is unavailable

Compare against a commit instead. Git object hashes cover content and history, so if you can obtain the expected commit hash through any channel you trust — a second mirror, a maintainer's post elsewhere, someone who cloned earlier — then:

```
git fetch <mirror> --tags
git rev-parse v1.2.3        # compare against the hash you trust
git verify-tag v1.2.3       # if the tag is signed
```

A mirror whose history matches a commit hash you trust from elsewhere is a faithful mirror.

Builds from other sources

If you have an `.ipa`, an `.apk`, or a Mac app from a forum, a chat group, a file locker, or any mirror, you cannot verify it, and this project cannot help you verify it. There is no published signing key for compiled builds and no reproducible-build pipeline, so there is nothing to compare a binary against.

What to do instead, in order of preference: install from the App Store; build from verified source; or, if neither is possible, treat that build as untrusted — assume anything you type into it may be disclosed, do not use it for anything sensitive, and do not carry it somewhere it being on your phone is itself a risk.

Do not rely on the app looking right. A modified build has no reason to look different.

For maintainers

Cutting a release:

- Push the tag. `source-manifest.yml` runs and attaches `SOURCE-MANIFEST.txt` to the release; if the release does not exist yet, collect the manifest from the workflow artifact and attach it when you publish.
- Sign the tag (`git tag -s`). A signed tag lets anyone verify the release came from a key you control, independent of GitHub. This needs a published key fingerprint to be useful — see the gap below.
- Note the commit hash somewhere outside this repository. If the repository is taken down, a hash recorded elsewhere is what lets people verify a mirror.

Known gaps, so nobody assumes more protection than exists:

- **No published signing key.** Tags are not currently verifiable against a known key. Publishing a fingerprint through channels independent of GitHub, and signing tags with it from then on, is the missing piece.
- **No verifiable compiled builds outside the App Store.** Closing this needs either a signed-and-notarized release pipeline or a reproducible build, and until one exists the guidance above stands.
- **No non-GitHub source mirror.** Every remote for this project is on the platform the takedown demands were served to. A mirror on independent infrastructure, published before it is needed, would mean a takedown does not remove the ability to verify.